

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA

**UNIVERSIDAD TECNOLÓGICA
NACIONAL**

**TRABAJO PRÁCTICO INTEGRADOR -
PROGRAMACIÓN 1**

Proyecto: Gestión de Datos de Países en Python

Docente:

- Cinthia Rigonni
- Ariel Enferrel

Integrantes:

- Gabriel Damian Vega: 39.768.858
- Roberto Moises Jaimes:

Fecha de Entrega: 18/06/2026

ÍNDICE GENERAL

1. Introducción (pág. 2)
2. Objetivo del proyecto (pág. 2)
3. Marco teórico (pág. 3,4 y 5)
 - 3.1 Listas
 - 3.2 Diccionarios
 - 3.3 Funciones
 - 3.4 Condicionales
 - 3.5 Estructuras repetitivas
 - 3.6 Ordenamientos
 - 3.7 Estadísticas básicas
 - 3.8 Archivos CSV
4. Descripción del problema (pág. 5)
5. Estructura de datos utilizada (pág. 6)
6. Decisiones técnicas y arquitectura (pág. 6)
7. Flujo general del sistema (pág. 6)
8. Funcionalidades implementadas (pág. 7)
9. Validaciones y manejo de errores (pág. 7)
10. Dificultades encontradas y resolución (pág. 7)
11. Conclusiones (pág. 7)
12. Bibliografía / Webgrafía (pág. 8)
13. Links de entrega (pág. 8)

1. INTRODUCCIÓN

En el presente Trabajo Práctico Integrador se expone el diseño, desarrollo e implementación de una aplicación de consola en Python destinada a la gestión integral de datos de países. El software ha sido estructurado para operar utilizando archivos en formato CSV (Comma Separated Values) como persistencia de datos secundaria, permitiendo realizar consultas interactivas, filtrados avanzados, ordenamientos algorítmicos y análisis estadísticos en tiempo real.

La arquitectura del sistema aplica los paradigmas fundamentales de la programación estructurada y modular aprendidos en la cátedra de Programación 1, promoviendo la legibilidad, la separación de responsabilidades y la robustez frente a fallos de entrada de datos.

2. OBJETIVO DEL PROYECTO

El propósito principal de este proyecto consiste en afianzar los conceptos técnicos del desarrollo de software mediante la resolución de un problema de dominio real. Los objetivos específicos técnico-académicos comprenden:

- **Procesamiento de Archivos:** Lectura, validación gramatical y escritura asíncrona/secuencial en archivos CSV.
- **Representación en Memoria:** Estructuración eficiente de tipos de datos compuestos (Listas de Diccionarios).
- **Manipulación de Datos:** Implementación de algoritmos de filtrado condicional y criterios de ordenación paramétrica (ascendente y descendente).
- **Generación de Métricas:** Deducción de indicadores estadísticos clave como promedios aritméticos y cálculo de extremos (máximos y mínimos).

3. MARCO TEÓRICO

3.1 Listas

Las listas en Python representan secuencias mutables y ordenadas de elementos. Permiten indexación base cero y dinamismo en su tamaño, lo cual facilita la inserción de nuevos registros mediante métodos nativos como `.append()`. En este sistema, la lista actúa como la colección principal en memoria que aloja la totalidad del dataset dinámico extraído del archivo externo.

3.2 Diccionarios

Un diccionario es una estructura asociativa basada en mapas de claves y valores (hashing tables). A diferencia de las secuencias numéricas, el acceso se realiza mediante claves de tipo cadena de texto (strings). Cada país se modela individualmente como un diccionario con claves explícitas (nombre, poblacion, superficie, continente), optimizando el acceso semántico a las propiedades de la entidad.

3.3 Funciones

La modularización fundamenta la división de un problema complejo en sub-problemas independientes de menor envergadura. Bajo el principio de responsabilidad única, cada función diseñada ejecuta un único proceso lógico determinado (ej. calcular promedio, validar enteros), reduciendo el acoplamiento y potenciando la reutilización del código fuente.

3.4 Condicionales

Las estructuras de control condicional (if-elif-else) posibilitan la bifurcación del flujo de ejecución del programa en respuesta a la evaluación de expresiones booleanas. Son el mecanismo central para la toma de decisiones algorítmicas, la validación de dominios y la bifurcación del menú principal.

3.5 Estructuras Repetitivas

Los bucles iterativos determinados (for) e indeterminados (while) permiten recorrer colecciones de datos y sostener ciclos de interacción indefinidos con el operador humano (como el mantenimiento activo del menú del sistema hasta la recepción explícita del comando de salida).

3.6 Ordenamientos

La organización sistemática de registros según métricas específicas es crítica en la ingeniería de software. Para este proyecto, se implementaron mecanismos de ordenación utilizando tanto lógica algorítmica iterativa de comparación como el uso avanzado de funciones internas parametrizadas con funciones anónimas lambda para la permutación ordenada de colecciones complejas.

3.7 Estadísticas Básicas

Se aplican operaciones matemáticas elementales sobre los conjuntos de datos para extraer conocimiento de valor agregado. Esto incluye la sumatoria y división para promedios de variables continuas, y comparaciones secuenciales para identificar extremos absolutos.

3.8 Archivos CSV

El estándar CSV almacena datos tabulares en texto plano. La utilización del módulo integrado de Python csv a través de clases de alto nivel como csv.DictReader y csv.DictWriter permite mapear de manera directa las filas del archivo con los diccionarios en memoria activa, preservando la integridad de las columnas.

3.9 Librería Questionary

La librería rich es utilizada extensivamente para mejorar la presentación visual en la consola, permitiendo:

- Paneles (Panel) para títulos y mensajes importantes.
- Tablas (Table) para mostrar listados de países y estadísticas de forma estructurada.
- Colores y Estilos (con console.print) para resaltar errores, advertencias e información.

Su funcionamiento se basa en una regla de tres pasos muy simple:

Importar `from questionary import` Configurar la pregunta `ask` Lanzar con `.ask()`.

El método `.ask()` detiene la ejecución del programa, muestra la interfaz interactiva en la terminal y devuelve la respuesta del usuario directamente como una variable de Python.

Para instalarlo se debe ejecutar en la terminal lo siguiente:

```
bash
pip install questionary
```

3.10 Librería rich

Rich es una librería de Python diseñada para escribir texto con estilo, color y componentes visuales avanzados en la terminal (CLI). A diferencia del texto plano tradicional, permite transformar la consola en una interfaz gráfica basada en texto (TUI) de forma extremadamente rápida.

Su funcionamiento es directo: sustituye o complementa las funciones de salida nativas de Python (como `print()`) por un sistema de renderizado que interpreta etiquetas de estilo e imprime componentes interactivos preconstruidos.

Para instalarlo se debe ejecutar en la terminal lo siguiente:

```
bash
pip install rich
```

3.11 Unicodedata

unicodedata es una librería nativa de Python (no requiere instalación) que da acceso a la Base de Datos de Caracteres Unicode (UCD). Funciona como un diccionario e inspector avanzado que contiene las propiedades ocultas de absolutamente cada letra, símbolo, emoji o número existente en el estándar informático mundial.

Su utilidad principal radica en la limpieza y estandarización de texto, la identificación de caracteres extraños y la conversión de símbolos a formatos legibles. Opera mediante funciones específicas que extraen metadatos de cualquier cadena de texto.

3.12 Funciones any y list Comprehensions

La función integrada any() toma un elemento ejecutable (como una lista o tupla) y devuelve True si al menos uno de los elementos es verdadero. Si todos son falsos (o la lista está vacía), devuelve False.

Por otra parte, la comprensión de listas es una forma elegante y compacta de crear nuevas listas a partir de otras listas o elementos iterables en una sola línea de código, reemplazando los tradicionales bucles for.

3.13 Variables Globales / Constantes

Una variable global en Python es aquella que se define fuera de cualquier función o clase, normalmente en el nivel superior del archivo (script). Esto significa que su alcance (o scope) abarca a todo el programa y puede ser leída desde cualquier lugar del código.

Se definen CARPETA_ACTUAL, RUTA_CSV, CAMPOS como constantes globales para facilitar la configuración y el acceso a datos compartidos.

4. DESCRIPCIÓN DEL PROBLEMA

El desafío planteado exige procesar un conjunto homogéneo de datos geográficos y demográficos propensos a errores de digitación o formatos de archivo inválidos. Se requiere proveer una interfaz limpia en consola mediante la cual un usuario no técnico sea capaz de interactuar con el dataset subyacente sin alterar la consistencia de los tipos de datos, garantizando la persistencia permanente una vez finalizada la sesión de trabajo.

5. ESTRUCTURA DE DATOS UTILIZADA

La arquitectura lógica elegida en memoria es una **Lista de Diccionarios**. A continuación, se detalla su representación estructural:

Estructura Contenedora	Elemento Interno Key(Clave)	Tipo de Dato	Ejemplo de Contenido (Values)
Lista (países: list)	nombre	String	"Argentina"
	poblacion	Entero	45376763
	superficie	Entero	2780400
	continente	String	"América"

6. DECISIONES TÉCNICAS Y ARQUITECTURA

El proyecto se estructuró de manera desacoplada siguiendo buenas prácticas de ingeniería:

- `main_paises.py`: Archivo de entrada del programa. Administra el bucle del menú principal y canaliza el flujo de entrada/salida.
- `funciones_paises.py`: Capa lógica de negocio y persistencia. Centraliza el procesamiento de datos, algoritmos, cálculos y acceso al sistema de archivos.
- `países.csv`: Origen y destino de datos persistidos.

Se optó por utilizar librerías de experiencia de usuario en consola como `rich` y `questionary` para robustecer estéticamente el entorno CLI y evitar errores de selección por parte del operador. Además, se implementa el uso de `unicodedata` para evitar errores de búsqueda y unificar lenguaje

7. FLUJO GENERAL DEL SISTEMA

El flujo operativo se rige bajo la siguiente secuencia cíclica:

1. Inicialización y comprobación de integridad del archivo `países.csv`.
2. Mapeo y parseo del CSV hacia la lista en memoria.
3. Despliegue del menú interactivo en consola.
4. Captura de selección y ejecución del módulo funcional correspondiente en la capa de lógica.
5. Actualización opcional de la memoria y posterior guardado en disco.
6. Re-evaluación del bucle de control (Salida / Continuación).

8. FUNCIONALIDADES IMPLEMENTADAS

De acuerdo a los requisitos exigidos en la consigna del TPI, el sistema cuenta con las siguientes funcionalidades operativas plenamente desarrolladas:

- Lectura automatizada del dataset base.
- Alta de nuevos países validando campos obligatorios.
- Modificación de variables cuantitativas (Población y Superficie) buscando por ID/Nombre.
- Buscador de texto con soporte para coincidencias parciales.
- Filtros combinados por rangos geográficos y demográficos.
- Módulo de ordenamiento flexible multifactorial.
- Tablero estadístico integral con promedios y distribución por continentes.
- Validaciones de duplicados y continentes reales existentes.

9. VALIDACIONES Y MANEJO DE ERRORES

Se implementó una estricta política de manejo de excepciones y validación preventiva:

- **Validación de Tipos:** Conversión segura mediante bloques try-except ValueError al procesar entradas numéricas.
- **Control de Nulos:** Bloqueo de inserciones con strings vacíos o compuestos únicamente por espacios en blanco.
- **Resiliencia del CSV:** Omisión controlada de filas corruptas o mal formadas en el archivo fuente, notificando al sistema para evitar caídas imprevistas.

10. DIFICULTADES ENCONTRADAS Y RESOLUCIÓN

Durante las pruebas de integración, la principal dificultad residió en coordinar el guardado inmediato en el archivo CSV cada vez que se alteraban los datos en memoria mediante operaciones de edición o inserción. Se solucionó centralizando las escrituras en una función modular de sincronización llamada inmediatamente tras confirmar una transacción exitosa.

Así mismo se encontraron dificultades a la hora de modificar la terminal de modo visual a la hora de interactuar con el usuario. Esto se solucionó mediante la implementación de librerías externas como **rich** y **questionary**, debiendo aprender su manera de implementación y instalando mediante el uso de **pip install**.

Por último, tuvimos que rever la validación de datos en los ingresos y búsquedas. Utilizando **unicodedata** para poder unificar el lenguaje y dando solución a este problema.

11. CONCLUSIONES

La realización de este proyecto integrador consolidó la comprensión práctica de las bases de la programación en Python. La correcta estructuración demostró ser una solución eficiente a la hora del desarrollo del código.

El correcto diseño previo de las estructuras de datos redujo la complejidad de los filtros y ordenamientos, evidenciando que una buena planificación antes de la codificación es efectiva para evitar repeticiones y simplificar las modificaciones en el proceso al tener localizados los bloques mediante la implementación de funciones.

12. BIBLIOGRAFÍA / WEBGRAFÍA

1. Campus UTN- Programación 1 Documentación brindada por la cátedra.
2. Encuentros sincrónicos
<https://campusvirtual.frm.utn.edu.ar/course/view.php?id=552§ion=64#tabs-tree-start>
3. <https://www.youtube.com/watch?v=ieJEJoBTXE8>
4. Colab IA
5. Gemini IA

13. ENLACES DE ENTREGA

Repositorio GitHub: <https://github.com/jaimesroberttto/tpi-jaimes-vega>

Video Demostrativo: